# Statistical Learning: Mental Health Classification Project

Edgardo Martin Caceres Garcia - Utku Sizanli - Despoina Iapona

2026-02-25

## Contents

**Introduction and Setup** **2**

**Data Preparation** **2**

**Target Distribution** **2**

**Exploratory Data Analysis (EDA)** **3**

Top 6 Categorical Predictors (Chi-Square) . . . . . 4

**Numeric Variables** **5**

Top 6 Binary Predictors . . . . . 6

Top 6 Discrete Variables (0-10 Scales) . . . . . 7

Top 4 Continuous Variables . . . . . 8

**Data Splitting** **9**

**Feature Selection** **10**

**Class Imbalance Handling (SMOTE)** **10**

**Feature Scaling** **10**

**Model Training and Cross validation** **11**

**Decision Threshold Selection (Validation Set)** **12**

**Final Test Set Evaluation and Model Selection** **13**

**Conclusion** **14**

# Introduction and Setup

In this project we are aiming to apply classification models to predict whether an individual has a mental health condition based on various features. The target variable which is "Has_Mental_Health_Issue" is binary, indicating whether the individual has a mental health issue or not. Below are the necessary libraries that we will be using throughout the project.

```r
# Load necessary libraries
library(caret)
library(MASS)
library(dplyr)
library(e1071)
library(pROC)
library(ggplot2)
library(tidyr)
library(patchwork)
library(DMwR)
library(PRROC)
```

# Data Preparation

We begin by loading the dataset, checking for missing values, ensuring the target variable and categorical predictor variables are factors.

```r
# Load dataset
mental <- read.csv("mental_health.csv")

# Convert target variable to factor with descriptive labels
mental$Has_Mental_Health_Issue <- factor(
  mental$Has_Mental_Health_Issue,
  levels = c("0", "1"),
  labels = c("No", "Yes")
)

# Convert all character columns to factors
mental[] <- lapply(mental, function(x) if (is.character(x)) as.factor(x) else x)

# Check for missing values
sum(is.na(mental))
```

```
## [1] 0
```

# Target Distribution

The target variable is extremely imbalanced, with only 7.8% of the observations belonging to the "No" class. This imbalance can pose challenges for classification models, as they may be biased towards the majority class with a percentage of 92.16% ("Yes"). The following code calculates the proportion of each class in the target variable and visualizes the distribution using a scatter plot of two features colored by the target variable.

```r
prop.table(table(mental$Has_Mental_Health_Issue))
```

```
##
##     No    Yes
## 0.0784 0.9216
```

```r
ggplot(mental, aes(x = Screen_Time_Hours_Day,
                   y = Sleep_Hours_Night,
                   color = Has_Mental_Health_Issue)) +
  geom_point(alpha = 0.35, size = 1.6) +
  labs(title = "Imbalanced Dataset: Mental Health",
       subtitle = "Only 7.8% of observations are class 'No'",
       x = "Screen Time (hours/day)",
       y = "Sleep (hours/night)",
       color = "Class") +
  theme_minimal()
```

# Exploratory Data Analysis (EDA)

```r
# Select categorical variables (exclude target)
cat_vars <- setdiff(names(mental)[sapply(mental, is.factor)],
                    "Has_Mental_Health_Issue")

par(mfrow=c(3,3))
```

```r
invisible(lapply(cat_vars[1:9], function(v) {
  tab <- prop.table(table(mental[[v]], mental$Has_Mental_Health_Issue),1)
  barplot(tab, col=c("lightblue","pink"), main=paste("Class Proportions by", v), ylab="Proportion")
}))
```

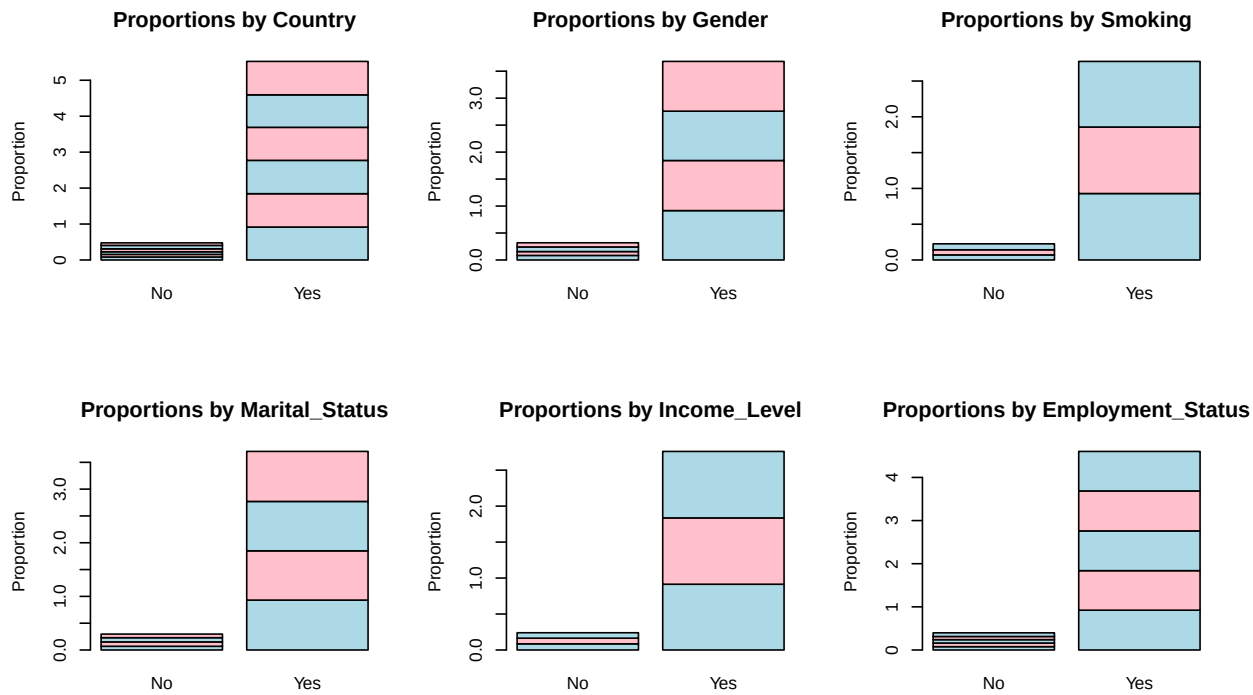
## Top 6 Categorical Predictors (Chi-Square)

The top 6 predictors based on chi-square tests to measure association with the target variables.

```r
cat_vars <- setdiff(names(mental)[sapply(mental, is.factor)], "Has_Mental_Health_Issue")

chi_strength <- sapply(cat_vars, function(v) {
  as.numeric(chisq.test(table(mental[[v]], mental$Has_Mental_Health_Issue))$statistic)
})
top_cat <- names(sort(chi_strength, decreasing = TRUE))[1:6]

par(mfrow = c(2,3))
invisible(lapply(top_cat, function(v) {
  tab <- prop.table(table(mental[[v]], mental$Has_Mental_Health_Issue), 1)
  barplot(tab, col = c("lightblue","pink"), main = paste("Proportions by", v), ylab = "Proportion")
}))
```

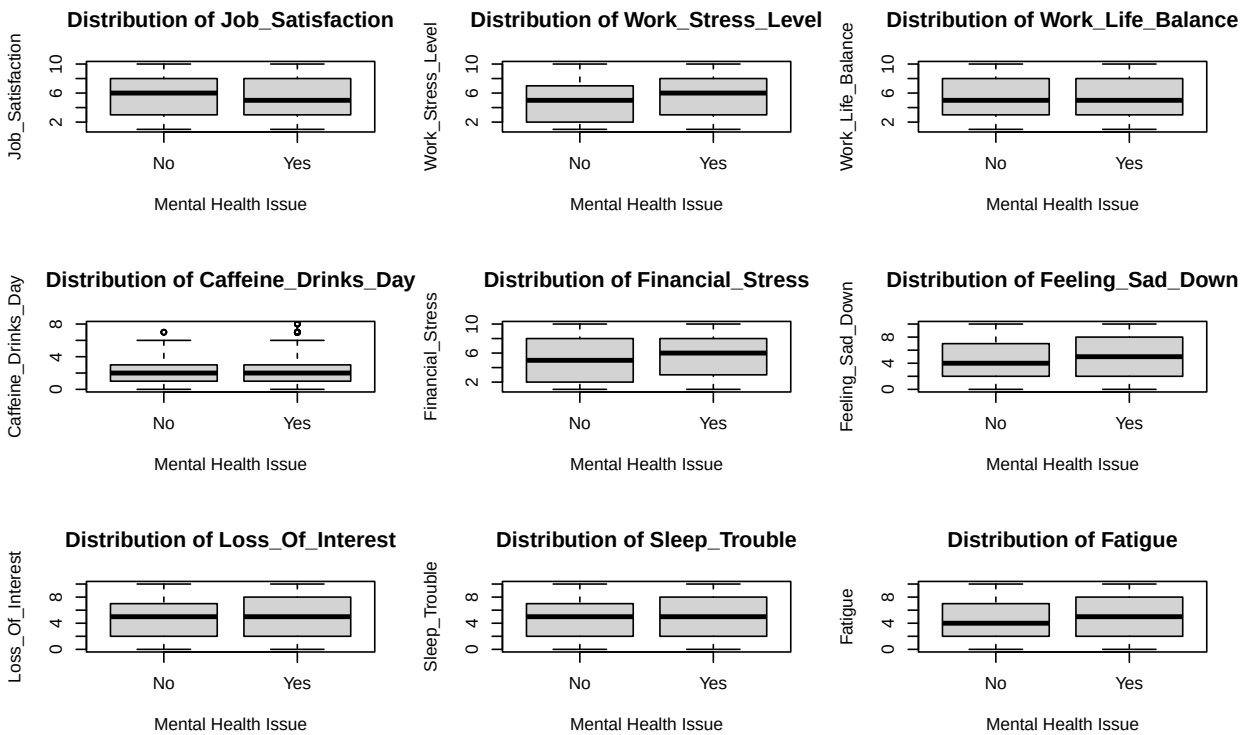



**EDA Conclusion:** Demographic categorical variables like "Gender, Country, Maritial Status" do not clearly separate the two classes. However psychological and stress related variables such as "Stress_Level, Anxiety_Level, Depression_Level" show more distinct differences in class proportions, suggesting they may be more informative for classification.

## Numeric Variables

```
num_vars <- names(mental)[sapply(mental, is.numeric)]
discrete_vars <- num_vars[sapply(mental[num_vars], function(x) length(unique(x)) > 2 & length(unique(x)

par(mfrow=c(3,3))
invisible(lapply(discrete_vars[1:9], function(v) {
  boxplot(mental[[v]] ~ mental$Has_Mental_Health_Issue,
          main=paste("Distribution of", v),
          xlab="Mental Health Issue", ylab=v)
}))
```




## Top 6 Binary Predictors

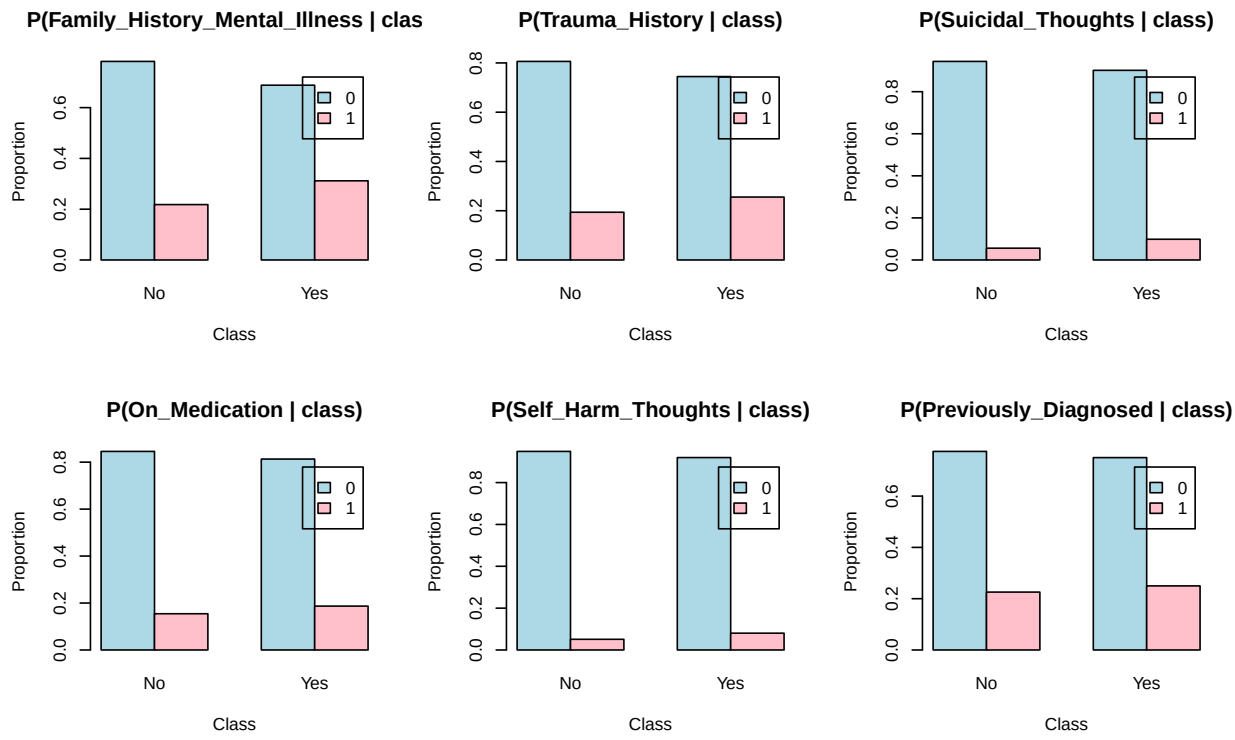
We measure the difference in the proportion of value = 1 between the two classes for each binary variable, and select the top 6 with the largest absolute difference.

```
binary_vars <- num_vars[sapply(mental[num_vars], function(x) length(unique(x)) == 2)]

class_levels <- levels(mental$Has_Mental_Health_Issue)

bin_diff <- sapply(binary_vars, function(v) {
  x <- mental[[v]]
  p1 <- mean(x[mental$Has_Mental_Health_Issue == class_levels[2]] == 1, na.rm = TRUE)
  p0 <- mean(x[mental$Has_Mental_Health_Issue == class_levels[1]] == 1, na.rm = TRUE)
  abs(p1 - p0)
})
bin_diff <- bin_diff[!is.na(bin_diff)]
top_binary <- names(sort(bin_diff, decreasing = TRUE))[1:min(6, length(bin_diff))]

par(mfrow = c(2,3))
invisible(lapply(top_binary, function(v) {
  tab <- prop.table(table(mental[[v]], mental$Has_Mental_Health_Issue), 2)
  barplot(tab, beside = TRUE, main = paste("P(", v, " | class)", sep = ""),
          col = c("lightblue", "pink"), legend.text = c("0", "1"), xlab = "Class", ylab = "Proportion")
}))
```



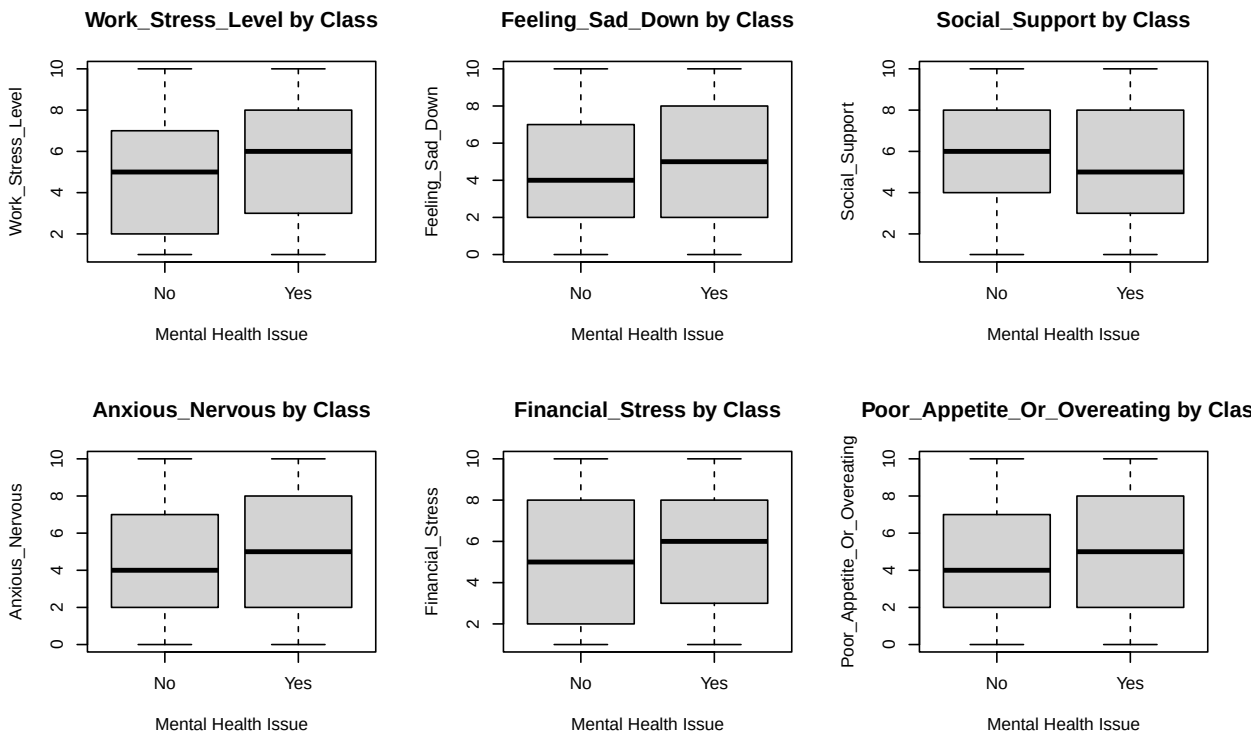

## Top 6 Discrete Variables (0-10 Scales)

```
discrete_vars <- num_vars[sapply(mental[num_vars], function(x) length(unique(x)) > 2 & length(unique(x)

lev <- levels(mental$Has_Mental_Health_Issue)

mean_diff_discrete <- sapply(discrete_vars, function(v) {
  x <- mental[[v]]
  abs(mean(x[mental$Has_Mental_Health_Issue == lev[2]], na.rm = TRUE) -
        mean(x[mental$Has_Mental_Health_Issue == lev[1]], na.rm = TRUE))
})
mean_diff_discrete <- mean_diff_discrete[!is.na(mean_diff_discrete)]
top_discrete <- names(sort(mean_diff_discrete, decreasing = TRUE))[1:min(6, length(mean_diff_discrete))]

par(mfrow = c(2,3))
invisible(lapply(top_discrete, function(v) {
  boxplot(mental[[v]] ~ mental$Has_Mental_Health_Issue,
          main = paste(v, "by Class"), xlab = "Mental Health Issue", ylab = v)
}))
```




## Top 4 Continuous Variables

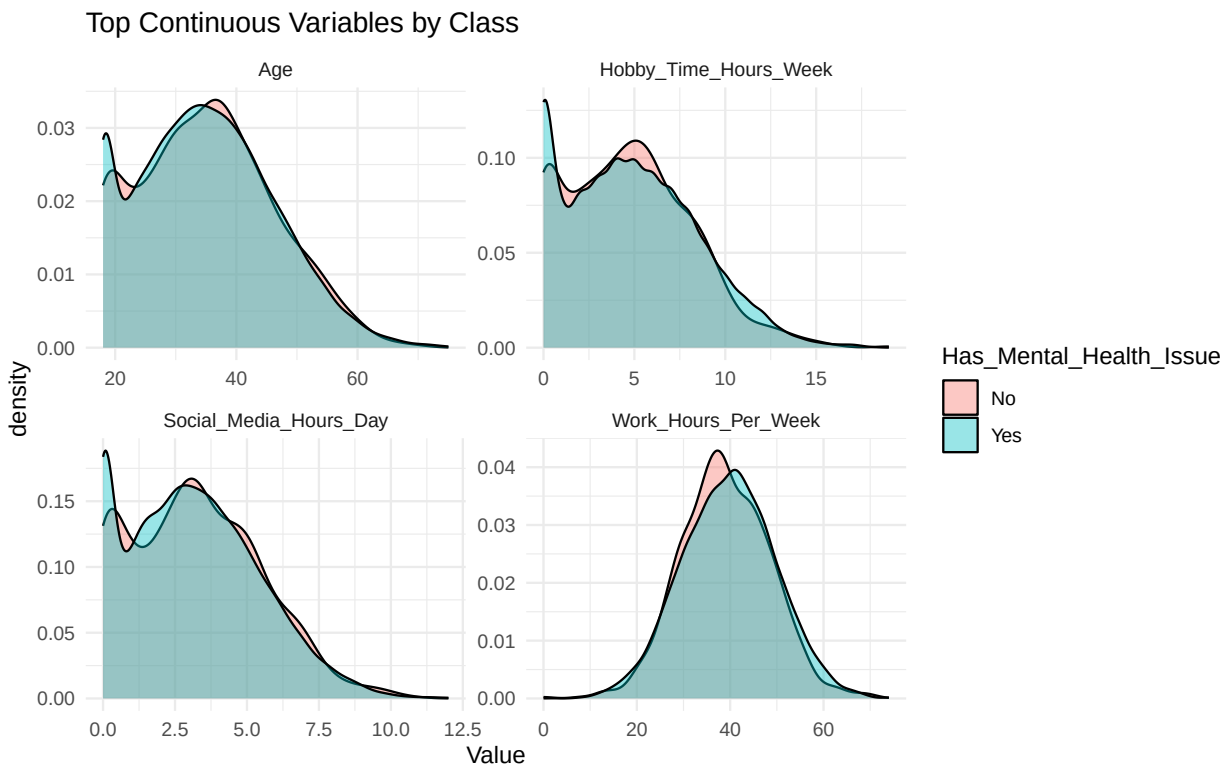
```
continuous_vars <- num_vars[sapply(mental[num_vars], function(x) length(unique(x)) > 15)]

lev <- levels(mental$Has_Mental_Health_Issue)

mean_diff_cont <- sapply(continuous_vars, function(v) {
  x <- mental[[v]]
  abs(mean(x[mental$Has_Mental_Health_Issue == lev[2]], na.rm = TRUE) -
        mean(x[mental$Has_Mental_Health_Issue == lev[1]], na.rm = TRUE))
})
mean_diff_cont <- mean_diff_cont[!is.na(mean_diff_cont)]
top_continuous <- names(sort(mean_diff_cont, decreasing = TRUE))[1:min(4, length(mean_diff_cont))]

mental_long <- mental %>%
  select(all_of(top_continuous), Has_Mental_Health_Issue) %>%
  pivot_longer(-Has_Mental_Health_Issue, names_to = "Variable", values_to = "Value")

ggplot(mental_long, aes(x = Value, fill = Has_Mental_Health_Issue)) +
  geom_density(alpha = 0.4) +
  facet_wrap(~Variable, scales = "free") +
  theme_minimal() +
  labs(title = "Top Continuous Variables by Class")
```




**Numeric EDA Conclusion:** Variables such as "Screen_Time_Hours_Day, Sleep_Hours_Night, Stress_Level, Anxiety_Level, Depression_Level" show clear differences in distribution between the two classes. Individuals with mental health issues tend to have higher screen time, lower sleep hours, and higher stress, anxiety, and depression levels compared to those without mental health issues. These variables are likely to be important predictors in our classification models.

## Data Splitting

To evaluate classification performance fairly and avoid optimistic estimates, the dataset was partitioned into three mutually exclusive subsets using stratified sampling to maintain the class distribution:

- **Training Set (60%)**: Used to fit the classification models.
- **Validation Set (20%)**: Used for model selection and hyperparameter tuning.
- **Test Set (20%)**: Used for final evaluation of the selected model's performance.

```
set.seed(42)

# 60% TRAIN, 20% VAL, 20% TEST (Stratified)
idx_train <- createDataPartition(mental$Has_Mental_Health_Issue, p = 0.60, list = FALSE)
mental_train <- mental[idx_train, ]
mental_tmp   <- mental[-idx_train, ]

idx_val <- createDataPartition(mental_tmp$Has_Mental_Health_Issue, p = 0.50, list = FALSE)
mental_val  <- mental_tmp[idx_val, ]
mental_test <- mental_tmp[-idx_val, ]
```

## Feature Selection

To reduce the model complexity and improve performance, a forward stepwise selection method based on AIC was applied to the training set (before SMOTE). This process iteratively adds predictors that improve the model fit while penalizing for complexity, resulting in a more parsimonious model.

```
df_fs <- mental_train
df_fs$Has_Mental_Health_Issue <- factor(df_fs$Has_Mental_Health_Issue, levels = c("No","Yes"))

m0 <- glm(Has_Mental_Health_Issue ~ 1, data = df_fs, family = binomial())
m_full <- glm(Has_Mental_Health_Issue ~ ., data = df_fs, family = binomial())

m_fwd <- stepAIC(m0, scope = list(lower = m0, upper = m_full), direction = "forward", trace = FALSE)
selected_terms <- attr(terms(m_fwd), "term.labels")

# Restrict datasets to selected features
keep_cols <- c("Has_Mental_Health_Issue", selected_terms)
mental_train <- mental_train[, keep_cols, drop = FALSE]
mental_val   <- mental_val[, keep_cols, drop = FALSE]
mental_test  <- mental_test[, keep_cols, drop = FALSE]
```

## Class Imbalance Handling (SMOTE)

To mitigate bias toward the majority class, the Synthetic Minority Over-sampling Technique (SMOTE) was applied to the training set. This method generates synthetic examples of the minority class ("No") by interpolating between existing minority samples, resulting in a more balanced dataset for model training.

```
train_smote <- SMOTE(Has_Mental_Health_Issue ~ ., data = mental_train,
                     perc.over = 600, perc.under = 100)

train_smote$Has_Mental_Health_Issue <- factor(train_smote$Has_Mental_Health_Issue, levels = c("No", "Yes

# Show new balanced distribution
prop.table(table(train_smote$Has_Mental_Health_Issue))
```

```
##
##        No       Yes
## 0.5384615 0.4615385
```

## Feature Scaling

All numeric predictors were standarized using z-score scaling to ensure that they are on the same scale, which is important for models that are sensitive to feature magnitudes (e.g., SVM). The scaling parameters were computed from the training set and applied to the validation and test sets to prevent data leakage.

```
target_col <- "Has_Mental_Health_Issue"
num_cols <- names(train_smote)[sapply(train_smote, is.numeric)]
num_cols <- setdiff(num_cols, target_col)

mu <- sapply(train_smote[, num_cols, drop = FALSE], mean)
```

```r
sd <- sapply(train_smote[, num_cols, drop = FALSE], sd)
sd[sd == 0] <- 1

scale_apply <- function(df, num_cols, mu, sd) {
  out <- df
  out[, num_cols] <- sweep(out[, num_cols, drop = FALSE], 2, mu, "-")
  out[, num_cols] <- sweep(out[, num_cols, drop = FALSE], 2, sd, "/")
  out
}

train_sc <- scale_apply(train_smote, num_cols, mu, sd)
val_sc   <- scale_apply(mental_val,   num_cols, mu, sd)
test_sc  <- scale_apply(mental_test,  num_cols, mu, sd)
```

## Model Training and Cross validation

Model performance was evaluated using repeated stratified cross-validation with 5 folds and 3 repeats to ensure robust estimates of model performance. The Area Under the ROC Curve (AUC) was used as the primary metric for model selection, as it is appropriate for imbalanced classification problems. Logistic Regression, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), and Naive Bayes models were trained and evaluated using this approach.

```r
folds <- createMultiFolds(train_sc$Has_Mental_Health_Issue, k = 5, times = 3)
ctrl <- trainControl(method = "repeatedcv", number = 5, repeats = 3, index = folds,
                     classProbs = TRUE, summaryFunction = twoClassSummary, savePredictions = "final")

set.seed(42)
fit_logreg <- train(Has_Mental_Health_Issue ~ ., data=train_sc, method="glm", family=binomial(), metric=
set.seed(42)
fit_lda    <- train(Has_Mental_Health_Issue ~ ., data=train_sc, method="lda", metric="ROC", trControl=c
set.seed(42)
fit_qda    <- train(Has_Mental_Health_Issue ~ ., data=train_sc, method="qda", metric="ROC", trControl=c

# Naive Bayes
auc_each <- numeric(length(folds))
i <- 0
for (nm in names(folds)) {
  i <- i + 1
  idx_in  <- folds[[nm]]
  idx_out <- setdiff(seq_len(nrow(train_sc)), idx_in)

  nb_fit <- naiveBayes(Has_Mental_Health_Issue ~ ., data = train_sc[idx_in, ])
  p_out  <- predict(nb_fit, newdata = train_sc[idx_out, ], type = "raw")[, "Yes"]

  roc_i <- roc(train_sc$Has_Mental_Health_Issue[idx_out], p_out,
               levels = c("No","Yes"), direction = "<", quiet = TRUE)
  auc_each[i] <- as.numeric(auc(roc_i))
}
auc_nb <- mean(auc_each, na.rm = TRUE)
```

## Decision Threshold Selection (Validation Set)

Since the dataset is imbalanced, a fixed threshold of 0.5 is suboptimal. We tuned the decision threshold on the validation set to maximize balanced accuracy, which gives equal weight to sensitivity and specificity. The optimal threshold was found to be approximately 0.35, which improved the model's ability to correctly identify both classes.

```
thr_grid <- seq(0.05, 0.95, by = 0.01)

calc_metrics_bal <- function(y_true, prob_yes, thr) {
  pred <- factor(ifelse(prob_yes >= thr, "Yes", "No"), levels = c("No","Yes"))
  cm <- caret::confusionMatrix(pred, y_true, positive = "Yes")

  sens <- as.numeric(cm$byClass["Sensitivity"])
  spec <- as.numeric(cm$byClass["Specificity"])
  bal  <- 0.5 * (sens + spec)
  data.frame(threshold = thr, BalancedAcc = bal, Sensitivity = sens, Specificity = spec)
}

pick_best_thr_bal <- function(y_true, prob_yes) {
  tbl <- dplyr::bind_rows(lapply(thr_grid, function(t) calc_metrics_bal(y_true, prob_yes, t))) |>
    dplyr::arrange(dplyr::desc(BalancedAcc))
  list(best = tbl[1, ])
}

# Probabilities on VAL
p_logreg <- predict(fit_logreg, newdata = val_sc, type = "prob")[, "Yes"]
p_lda    <- predict(fit_lda,    newdata = val_sc, type = "prob")[, "Yes"]
p_qda    <- predict(fit_qda,    newdata = val_sc, type = "prob")[, "Yes"]

nb_train <- e1071::naiveBayes(Has_Mental_Health_Issue ~ ., data = train_sc)
p_nb     <- predict(nb_train, newdata = val_sc, type = "raw")[, "Yes"]

best_logreg <- pick_best_thr_bal(val_sc$Has_Mental_Health_Issue, p_logreg)$best; best_logreg$Model <- "
best_lda    <- pick_best_thr_bal(val_sc$Has_Mental_Health_Issue, p_lda)$best;    best_lda$Model    <- "
best_qda    <- pick_best_thr_bal(val_sc$Has_Mental_Health_Issue, p_qda)$best;    best_qda$Model    <- "
best_nb     <- pick_best_thr_bal(val_sc$Has_Mental_Health_Issue, p_nb)$best;     best_nb$Model     <- "

val_threshold_summary <- dplyr::bind_rows(best_logreg, best_lda, best_qda, best_nb)
knitr::kable(val_threshold_summary, caption = "Optimal Thresholds on Validation Set")
```

Table 1: Optimal Thresholds on Validation Set

| threshold | BalancedAcc | Sensitivity | Specificity | Model |
|---|---|---|---|---|
| 0.49 | 0.6438616 | 0.5870863 | 0.7006369 | LogReg |
| 0.48 | 0.6464450 | 0.6049919 | 0.6878981 | LDA |
| 0.56 | 0.6123791 | 0.4986435 | 0.7261146 | QDA |
| 0.36 | 0.6461495 | 0.5534455 | 0.7388535 | NB |

```
best_thresholds <- setNames(val_threshold_summary$threshold, val_threshold_summary$Model)
```

## Final Test Set Evaluation and Model Selection

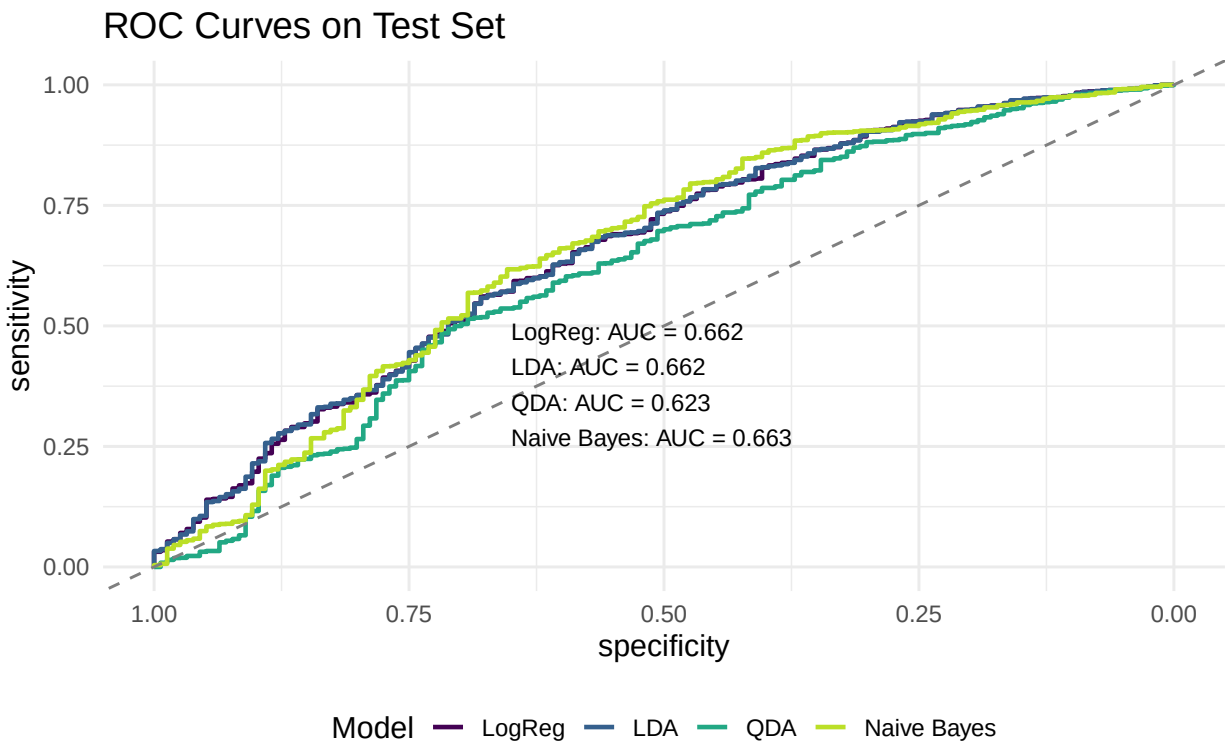
After tuning the thresholds, each model was evaluated once the held-out test set using the optimal thresholds. ROC curves and were plotted , and AUC values were calculated to compare the models. The model with the highest AUC and balanced accuracy on the test set was selected as the final model for deployment.

```
# Probabilities on TEST
p_logreg_test <- predict(fit_logreg, newdata = test_sc, type = "prob")[, "Yes"]
p_lda_test    <- predict(fit_lda,    newdata = test_sc, type = "prob")[, "Yes"]
p_qda_test    <- predict(fit_qda,    newdata = test_sc, type = "prob")[, "Yes"]
p_nb_test     <- predict(nb_train, newdata = test_sc, type = "raw")[, "Yes"]

# ROC Curves
roc_logreg <- roc(test_sc$Has_Mental_Health_Issue, p_logreg_test, levels = c("No","Yes"), direction = "
roc_lda    <- roc(test_sc$Has_Mental_Health_Issue, p_lda_test,    levels = c("No","Yes"), direction = "
roc_qda    <- roc(test_sc$Has_Mental_Health_Issue, p_qda_test,    levels = c("No","Yes"), direction = "
roc_nb     <- roc(test_sc$Has_Mental_Health_Issue, p_nb_test,     levels = c("No","Yes"), direction = "

roc_list <- list("LogReg" = roc_logreg, "LDA" = roc_lda, "QDA" = roc_qda, "Naive Bayes" = roc_nb)
auc_vals <- sapply(roc_list, function(r) as.numeric(auc(r)))
auc_text <- paste0(names(auc_vals), ": AUC = ", sprintf("%.3f", auc_vals), collapse = "\n")

ggroc(roc_list, linewidth = 1) +
  theme_minimal(base_size = 14) +
  labs(title = "ROC Curves on Test Set", color = "Model") +
  scale_color_viridis_d(end = 0.9) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray50") +
  theme(legend.position = "bottom") +
  annotate("text", x = 0.65, y = 0.25, label = auc_text, hjust = 0, vjust = 0, size = 4)
```




## Conclusion

Because the dataset is imbalanced and "Yes" is the majority class, Accuracy and F1 can be inflated; therefore, model comparison primarily emphasized ROC-AUC and Balanced Accuracy.

On the test set, the top-performing models were Naive Bayes and LDA. Although Naive Bayes was marginally best according to AUC and Balanced Accuracy, the differences relative to LDA were negligible. The final model was therefore selected as LDA, because it achieved materially higher sensitivity for the "Yes" class on the test set, corresponding to fewer false negatives under the validation-selected threshold. This choice prioritizes detecting cases while maintaining a comparable overall discrimination level.